Autoscaling proactif et conscient des dépendances pour microservices

Propagation de charge sur le graphe d'appels et allocation sous contrainte de SLO

Présentation de
LEUDJEU WOUAPPI Beautrel Horssel
Matricule: 22U2079

Encadré par
Pr. DJAM Xaveria Youh Kimbi

Université de Yaoundé I
Faculté des Sciences
Département d'Informatique

6 novembre 2026

Plan de la présentation

1. Concepts clés
2. Contexte
3. Problématique
4. Revue de la littérature
5. La question que nous posons

Concepts clés du thème

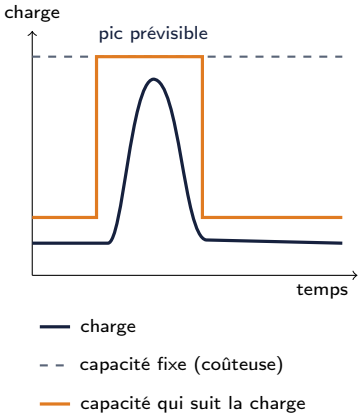
Microservices	Petits services autonomes qui coopèrent en s'appelant les uns les autres.
Instance	Copie d'un service en cours d'exécution ; plus il y en a, plus le service absorbe de charge.
Autoscaling	Ajustement automatique du nombre d'instances d'un service selon la charge.
Réactif / proactif	Agir après la saturation, ou anticiper à partir de la charge prévue.
SLO	Objectif de niveau de service, par exemple 95 % des requêtes en moins de 300 ms.
Graphe de dépendances	Relations d'appel entre services : un pic en amont se propage en aval.
Propagation de charge	Charge reçue par un service du fait de la charge de ses appelants.

Contexte général

- Les applications modernes (paiement mobile, commerce en ligne, billetterie...) sont découpées en **microservices** qui s'appellent en chaîne.
- Leur **charge varie fortement** : jours de paie, fêtes, promotions, ouverture de ventes...
- Pour s'adapter, on ajuste automatiquement le nombre d'instances de chaque service : c'est l'**autoscaling**.
- Les outils courants (comme le HPA de Kubernetes) décident **service par service**, avec des **seuils fixes**, et **après** la saturation.

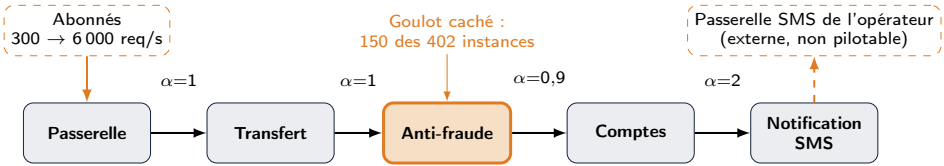
Enjeu

Respecter le **SLO** sans payer des ressources inutiles.



Comprendre avec un exemple : Mobile Money le jour de paie

Scénario fictif (chiffres illustratifs) : le jour de paie, à **8h00**, la charge passe de **300 à 6 000 requêtes/s**.



	Passerelle	Transfert	Anti-fraude	Comptes	Notification SMS
Charge λ_i (req/s)	6 000	6 000	6 000	5 400	10 800
Débit μ_i par instance	500	100	40	60	120
Instances n_i à 8h00	**12**	**60**	**150**	**90**	**90**
Instances en temps normal	1	3	8	5	5

α_{ij} : appels de i vers j par requête reçue • $\lambda_j = \sum_i \alpha_{ij}\,\lambda_i$ • $n_i = \lceil \lambda_i / \mu_i \rceil$

Classique (réactif, service par service)

Chaque service sature à son tour : environ **10 min** d'indisponibilité, aggravées par les relances des utilisateurs.

Intelligent (proactif, sur tout le graphe)

7h55 : les **402 instances** (au lieu de 22) démarrent ensemble : **0 min** d'indisponibilité.

Pourquoi l'autoscaling réactif échoue : trois limites structurelles

1. **Réactivité tardive.** Le service n'agit qu'une fois la saturation constatée. Or, à 8h00, la charge est déjà là, et le démarrage de nouvelles instances (*cold start*) prend encore du temps.
2. **Cécité aux dépendances.** Chaque service (transfert, anti-fraude, comptes...) a son propre autoscaler. La passerelle SMS est gérée par un opérateur externe. **Personne ne décide pour la chaîne entière**.
3. **Seuils fixes.** L'autoscaler applique une cible d'utilisation CPU ou mémoire choisie une fois pour toutes, sans savoir que le pic de paie est **prévisible** (il revient chaque mois) ni que la contrainte réelle est la latence de bout en bout.

Conséquence

La latence de bout en bout dépend de *tous* les composants simultanément, mais chaque autoscaler ne voit qu'un composant isolé — et parfois ne voit même pas les dépendances dont il a besoin pour agir.

Notre problématique

Problématique

Décider automatiquement, **avant** l'arrivée d'un pic de charge, du nombre d'instances de **chaque** microservice, en tenant compte des **appels entre services**, pour respecter le SLO au moindre coût.

Formellement : pour tout $i \in S$ et tout t, trouver $n_i(t) \in \mathbb{N}$ minimisant

$$\sum_{t} \sum_{i \in S} c_i \, n_i(t)$$

sous :

- $\forall t,\ L(t) \leq \mathrm{SLO}$;
- $\forall j \in S,\ \lambda_j(t) = \lambda_j^{ext}(t) + \sum_{(i,j) \in E} \alpha_{ij} \, \lambda_i(t)$;
- $\forall i \in S,\ \lambda_i(t) \leq n_i(t) \, \mu_i$.

où $n_i(t)$ est le nombre d'instances du service i, c_i son coût unitaire, $\lambda_i(t)$ sa charge, μ_i le débit d'une instance, α_{ij} le facteur d'appel de i vers j et $L(t)$ la latence de bout en bout.

Ce que la recherche a commencé à proposer

Notre problématique (la même pour chaque article)

Décider automatiquement, avant un pic de charge, du nombre d'instances de chaque microservice, en tenant compte des appels entre services, pour respecter le SLO au moindre coût.

Article	Auteurs (premiers)	Publication	Idée générale
Erms	S. Luo, H. Xu, K. Ye et al.	ASPLOS 2023 (conférence)	Gérer les ressources de microservices partagés en garantissant le SLA
GRAF	J. Park, B. Choi, C. Lee, D. Han	CoNEXT 2021 (conférence)	Allouer le CPU de façon proactive grâce à un réseau de neurones sur graphes
Mondal et al.	S. K. Mondal, X. Wu, H. M. D. Kabir et al.	Mathematics (MDPI), 2023	Prévoir la charge (GRU) et construire un autoscaler Kubernetes personnalisé

Article 1 : Erms (ASPLOS 2023) — méthodologie et limites

Fiche de l'article

Erms : Efficient Resource Management for Shared Microservices with SLA Guarantees

Auteurs : S. Luo, H. Xu, K. Ye, G. Xu, L. Zhang, J. He, G. Yang, C. Xu (Shenzhen Institute of Advanced Technology, University of Macau, Alibaba)

Résultat clé : violations de SLA divisées par 5 et ressources divisées par 1,6 par rapport à l'état de l'art.

Limites

- Graphes de dépendances supposés **statiques** ; sur-provisionnement s'ils sont très dynamiques (auteurs).
- Profilage hors ligne long : près de 3 jours pour un service.
- Dimensionnement à partir de la charge **observée** : pas de prévision (notre lecture).

Méthodologie

1. **Profilage hors ligne :** la latence d'un microservice est modélisée comme une fonction linéaire par morceaux de la charge (et de l'interférence CPU/mémoire).
2. **Fusion du graphe :** les appels parallèles sont ramenés à une chaîne séquentielle.
3. **Cibles de latence :** calcul par optimisation convexe (solution en forme fermée).
4. **Priorités :** ordonnancement par priorité aux microservices partagés, puis placement des conteneurs sur Kubernetes.
5. **Évaluation :** DeathStarBench (cluster de 20 machines) et simulations avec des traces d'Alibaba.

Article 2 : GRAF (CoNEXT 2021) — méthodologie et limites

Fiche de l'article

GRAF : A Graph Neural Network based Proactive Resource Allocation Framework for SLO-Oriented Microservices

Auteurs : J. Park, B. Choi, C. Lee, D. Han (KAIST, Toyota Motor Corporation)

Résultat clé : jusqu'à 19 % de CPU économisé ; lors d'un pic, 36 % de ressources en moins et une latence stabilisée jusqu'à 2,6 fois plus vite.

Limites

- **Passage à l'échelle** : le modèle peut se dégrader pour des centaines ou milliers de microservices (auteurs).
- Modèle entraîné **par application**, à refaire si elle change (environ 50 000 échantillons).
- Réagit à la charge **actuelle** du front-end : pas de prévision (notre lecture).

Méthodologie

1. **Observer** la charge du front-end et les traces des requêtes.
2. **Répartir** cette charge sur chaque microservice du graphe.
3. **Prédire** la latence de bout en bout (99e percentile) avec un réseau de neurones sur graphes (GNN).
4. **Optimiser** les quotas CPU par descente de gradient sous contrainte de SLO.
5. **Déployer** le nombre d'instances correspondant sur Kubernetes.
6. **Évaluation :** Online Boutique et Social Network (7 machines), face à l'autoscaler de Kubernetes réglé finement.

Article 3 : Mondal et al. (Mathematics 2023) — méthodologie et limites

Fiche de l'article

Toward Optimal Load Prediction and Customizable Autoscaling Scheme for Kubernetes

Auteurs : S. K. Mondal, X. Wu, H. M. D. Kabir, H.-N. Dai, K. Ni, H. Yuan, T. Wang (Macau University of Science and Technology et autres)

Résultat clé : le GRU est le meilleur modèle de prévision ; sur charge stable, le comportement est proche du HPA (faisabilité).

Limites

- Modèle construit sur l'utilisation CPU d'un nœud, pas par tâche (auteurs).
- Un modèle supervisé à entraîner par métrique (auteurs).
- Un seul déploiement à la fois : **pas de dépendances** entre services (notre lecture).

Méthodologie

1. **Comparer** ARIMA, LSTM, BiLSTM et GRU pour prédire la charge CPU (jeu de données Google cluster-data-2011).
2. **Retenir** le meilleur modèle : le GRU.
3. **Intégrer** le modèle dans un autoscaler personnalisé (Custom Pod Autoscaler) : collecte des métriques, puis évaluation du nombre de réplicas ; repli sur la logique du HPA si l'historique est trop court.
4. **Déployer** sur un cluster Kubernetes (1 maître, 2 travailleurs) et comparer au HPA.

Synthèse : ce que font les trois articles

	Erms	**GRAF**	**Mondal et al.**
Dépendances entre services	Oui (graphe d'appels)	Oui (GNN sur le graphe)	Non (un déploiement)
Anticipation	Non (charge observée)	Partielle (charge du front-end)	Oui (prévision GRU)
Objectif	SLA au moindre coût	CPU minimal sous SLO	Remplacer le HPA réactif
Limite principale	Graphes statiques ; profilage long	Passage à l'échelle ; réentraînement	Pas de dépendances ; validation limitée

Premier constat (à confirmer par une lecture approfondie)

Parmi ces trois articles, aucun ne combine la **prévision de la charge** et sa **propagation sur le graphe de dépendances**. Ces travaux montrent que la communauté a identifié le problème. Mais ils laissent ouverte la question de la propagation explicite de la charge prévue sur le graphe, et de la prise en compte des dépendances dont l'autoscaler dépend lui-même.

Le thème et la question de recherche

Thème

Autoscaling proactif et conscient des dépendances pour les architectures microservices : propagation de charge sur le graphe d'appels et allocation de ressources sous contrainte de SLO

Question de recherche (provisoire)

Comment dimensionner dynamiquement et simultanément l'ensemble des services d'une application microservices, en exploitant le graphe de dépendances et la prédiction de charge, de façon à respecter les SLO au moindre coût, y compris lorsque les dépendances nécessaires au provisionnement lui-même ne sont pas explicitement modélisées ?

Sous-questions de recherche

SQ1 Comment **modéliser la propagation de la charge** d'un service vers ses dépendances sur le graphe d'appels, y compris vers une dépendance non modélisée explicitement (par exemple une passerelle SMS externe) ?

SQ2 Comment **prévoir la charge d'entrée** de façon fiable, y compris face aux pics imprévus ?

SQ3 Comment déterminer **simultanément** le nombre d'instances de chaque service, en tenant compte de leur temps de démarrage, au moindre coût ?

Remarque

Ces questions guideront la lecture approfondie des articles ; la démarche de recherche sera précisée ensuite.

Merci de votre attention.